Salford & Co

TRACT+

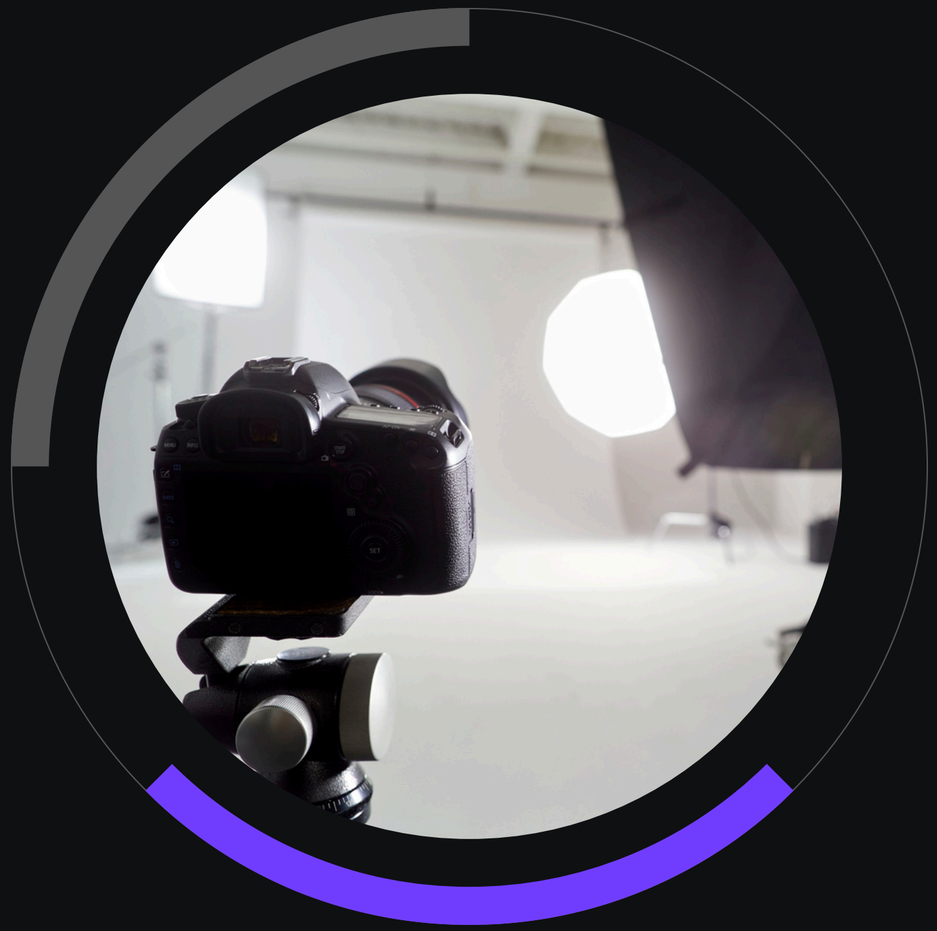
TWITTER
REPORT
ABUSE
CASSIFICATION
TEST

- Team Members:

1. Asmaa Ahmed Maryah	811634546	IS
2. Esraa Saber Ahmed Hafez	812555812	IS
3. Hagar Tarek abo elhassan	812501748	AI
4. Nesma Khairy El-Ghalban	811668583	IS
5. Zainab Sherif Talaat Mosad	812574337	AI

Problem

TRACT (Tweets Reporting Abuse Classification Task) is a specialized NLP project focused on identifying and classifying reports of abuse on Twitter. In an era where victims—especially teenagers and minorities—often turn to social media as a 'cry for help' rather than official authorities, this project aims to bridge the gap using Machine Learning. By classifying tweets into Abuse Reporting, Empathy, or General content, we build a system that can detect suffering and facilitate digital support in real-time



IMPACT & DATA

Dataset

Salford & Co

- Dataset Key Facts
- Core Task: An annotated corpus specifically designed for the task of classifying reports of abuse on Twitter.
- Source: Compiled by researcher Saichethan Miriyala Reddy and first published in 2020 via Mendeley Data and Kaggle.
- Labeling System: The data is categorized into three primary classes:
- Label 1: Reporting or explicit confession of abuse.
- Label 2: Expressions of empathy toward victims.
- Label 3: General content that does not fall into the above categories.
-

Turning Every Shot
into a Smart Capture

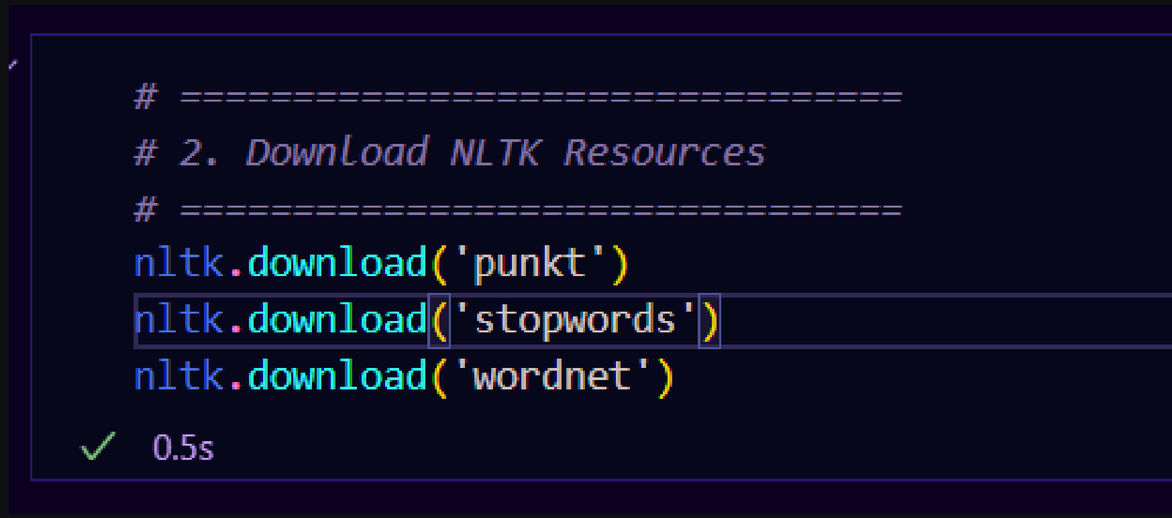
Libraries

```
# ==================================
# 1. Import Libraries
# ==================================
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
import nltk
import numpy as np
import torch
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from scipy.sparse import hstack
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from transformers import BertTokenizer, BertModel
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

10.7s

Natural Language Toolkit (NLTK) Setupmet

- Tokenization (Punkt): Breaking down tweets into individual words for processing.
- Stopwords Removal: Filtering out common words (e.g., "is", "the") to focus on high-impact terms.
- Lemmatization (Wordnet): Reducing words to their root form (e.g., "reporting" becomes "report") to improve model consistency.



Load Dataset

```
# ================================
# 3. Load Dataset
# ================================

import pandas as pd

# Load training dataset
train_df = pd.read_csv('train.tsv', sep='\t')

# Load validation dataset
validation_df = pd.read_csv('validation.tsv', sep='\t')
```

✓ 0.0s

Handle Missing Values

Garbage In, Garbage Out

We performed a strict filtering process to remove any incomplete records from both 'tweet' and 'label' columns.

```
# ================================
# 4. Handle Missing Values
# ================================

# Clean the training data
train_df.dropna(subset=['tweet', 'label'], inplace=True)

# Clean the validation data
validation_df.dropna(subset=['tweet', 'label'], inplace=True)

# Optional: Verify the results
print(f"Train size: {len(train_df)}")
print(f"Validation size: {len(validation_df)}")
```

✓ 0.0s

```
Train size: 2999
Validation size: 1500
```

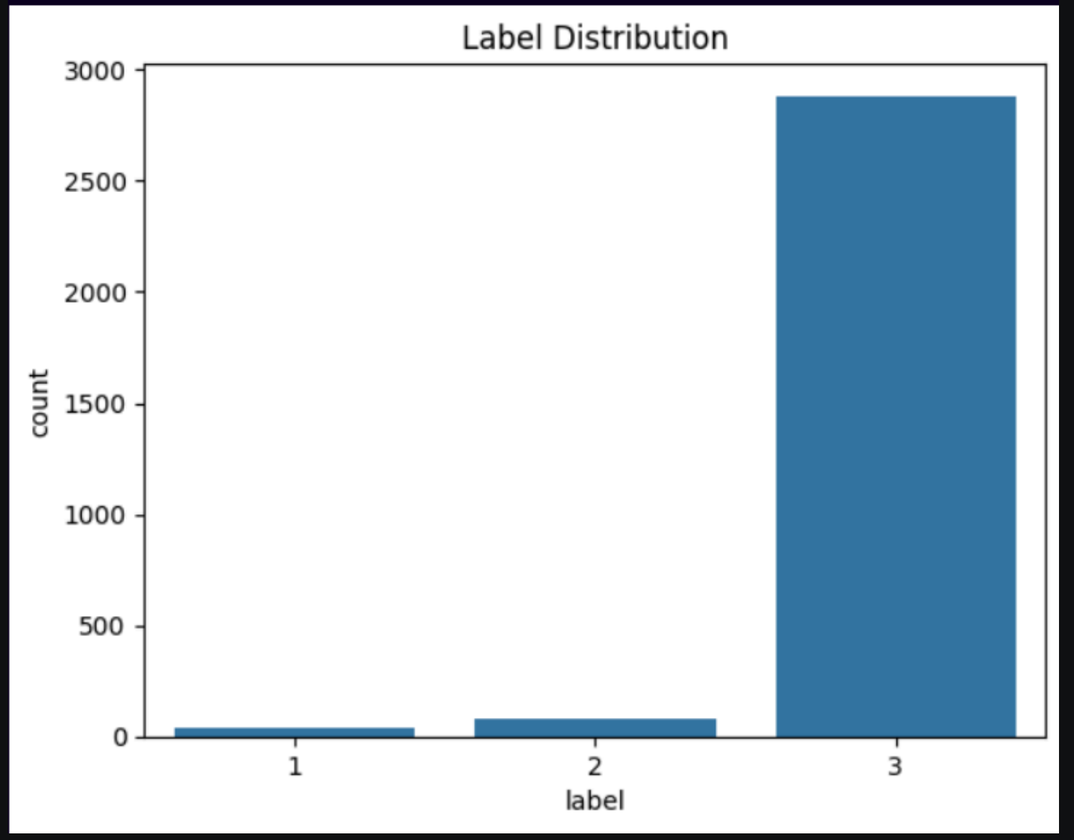
TEXT CLEANING FUNCTION

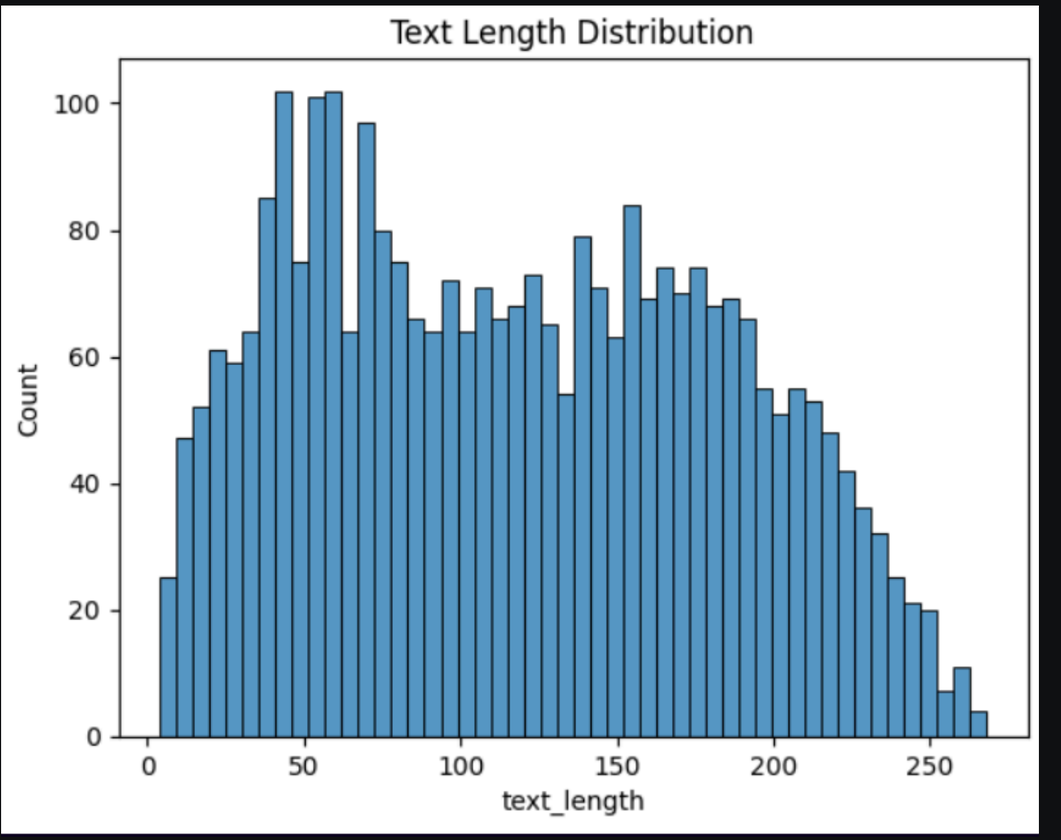
1. Noise Reduction
2. Text Normalization
3. Feature Refinement

```python
# ============================================================
# 5. TEXT CLEANING FUNCTION
# ============================================================
def clean_text(text):
    """
    Clean raw text by:
    - Lowercasing
    - Removing URLs
    - Removing special characters
    - Removing extra spaces
    """
    text = str(text).lower()
    text = re.sub(r'http\S+', '', text)          # Remove URLs
    text = re.sub(r'[^a-zA-Z\s]', '', text)      # Keep only letters , Remove specila character
    text = re.sub(r'\s+', ' ', text).strip()     # Remove extra spaces
    return text


# Apply cleaning
train_df['processed_text'] = train_df['tweet'].apply(clean_text)
validation_df['processed_text'] = validation_df['tweet'].apply(clean_text)
```

✓ 0.0s

Data Visualization





Term Frequency-Inverse Document Frequency

- Vectorization: Combined TF-IDF (statistical weights) with Custom Keywords (domain expertise) to capture the most critical signals of abuse.
- Optimization: Focused on 5,000 top features and Bigrams to ensure the model understands both word frequency and context (e.g., "domestic violence").

```
# ============================================================
# 6. TF-IDF + KEYWORD FEATURES
# ============================================================
# Convert text into numerical features using TF-IDF
tfidf = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.9
)

X_tfidf = tfidf.fit_transform(train_df['processed_text'])

# Define domain-specific abuse keywords
abuse_keywords = [
    "hit", "beaten", "abuse", "hurt", "injury",
    "bruise", "bleeding", "cry", "fear",
    "violence", "neglect", "pain"
]

def extract_keyword_features(text):
    """
    Create binary features indicating presence of abuse-related keywords
    """
    return [int(word in text) for word in abuse_keywords]

# Convert keyword features to array
kw_features = np.array(
    train_df['processed_text'].apply(extract_keyword_features).tolist()
)

# Combine TF-IDF features with keyword features
X_combined = hstack([X_tfidf, kw_features])
```

✓ 0.2s

Model Training & Evaluation Strategy

- Data Partitioning: Split data into 80% training and 20% testing, using Stratified Sampling to maintain class balance across labels.

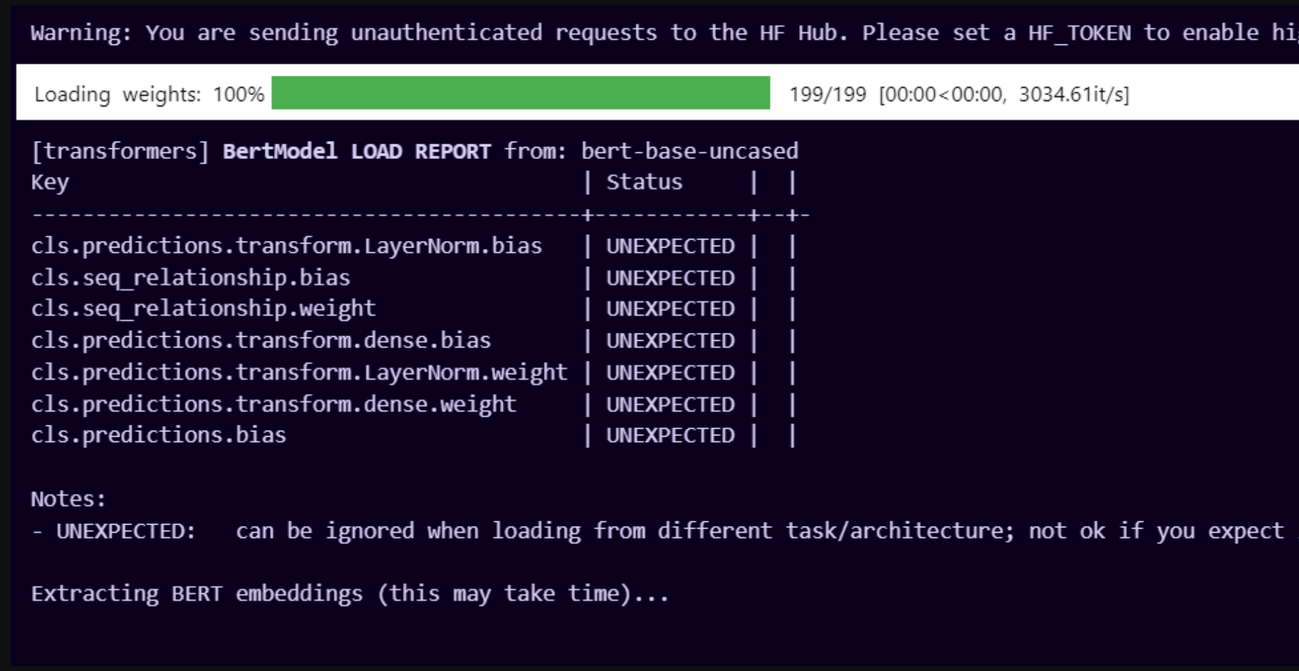
- Model Selection: Employed Logistic Regression for text classification, optimized with 1,000 iterations to ensure model convergence and accuracy.

```
# ============================================================
# 7. TRAIN-TEST SPLIT (TF-IDF MODEL)
# ============================================================
y = train_df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X_combined,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Train Logistic Regression model
tfidf_model = LogisticRegression(max_iter=1000)
tfidf_model.fit(X_train, y_train)

# Predictions
tfidf_predictions = tfidf_model.predict(X_test)
```

✓ 0.1s

Advanced Feature Extraction: BERT

- Contextual Embeddings: Leveraged a pre-trained BERT model to extract CLS embeddings, capturing deep semantic meanings and context from tweets.
- Computational Efficiency: Applied truncation and padding for standardized input, processing a representative subset to evaluate the power of Transformer-based models on the TRACT Corpus.



Model Evaluation Framework

- Multi-Metric Assessment: Evaluated model performance using Accuracy, Precision, Recall, and F1-Score to ensure a balanced judgment of classification quality.

- Diagnostic Tools: Leveraged Confusion Matrices and Detailed Classification Reports to identify specific areas where the model excels or requires further tuning.

```python
# ============================================================
# 9. MODEL EVALUATION FUNCTION
# ============================================================
def evaluate_model(y_true, y_pred, model_name="Model"):
    """
    Evaluate classification model using multiple metrics
    (supports multiclass classification)
    """
    print(f"\n{'='*20} {model_name} {'='*20}")

    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, average='weighted', zero_division=0)
    rec = recall_score(y_true, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_true, y_pred, average='weighted', zero_division=0)

    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {prec:.4f} (weighted)")
    print(f"Recall   : {rec:.4f} (weighted)")
    print(f"F1 Score : {f1:.4f} (weighted)")

    print("\nClassification Report:")
    print(classification_report(y_true, y_pred, zero_division=0))

    print("Confusion Matrix:")
    print(confusion_matrix(y_true, y_pred))
```

✓ 0.0s

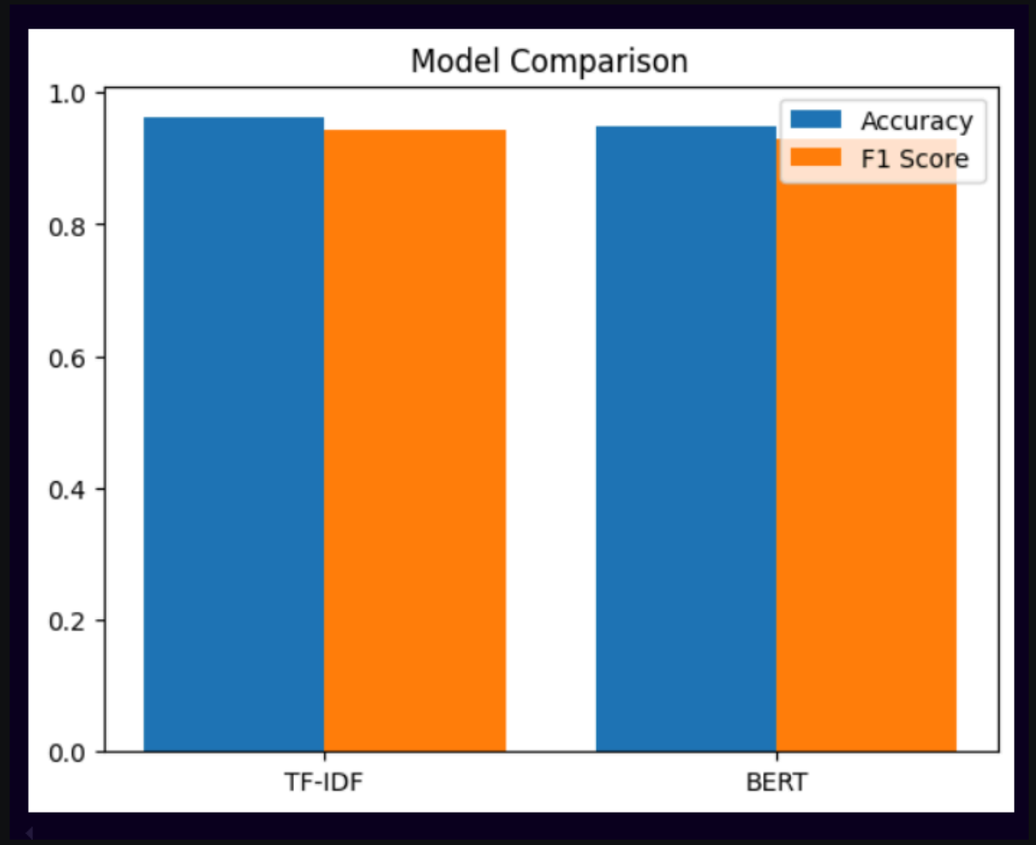
Results & Performance Analysis

- High Benchmark: Achieved an overall accuracy of 96% using TF-IDF, demonstrating strong performance on the majority class (General tweets).
- Model Comparison: While TF-IDF showed higher overall accuracy, BERT displayed better potential in identifying minority classes (Abuse/Empathy) even with a smaller training subset.

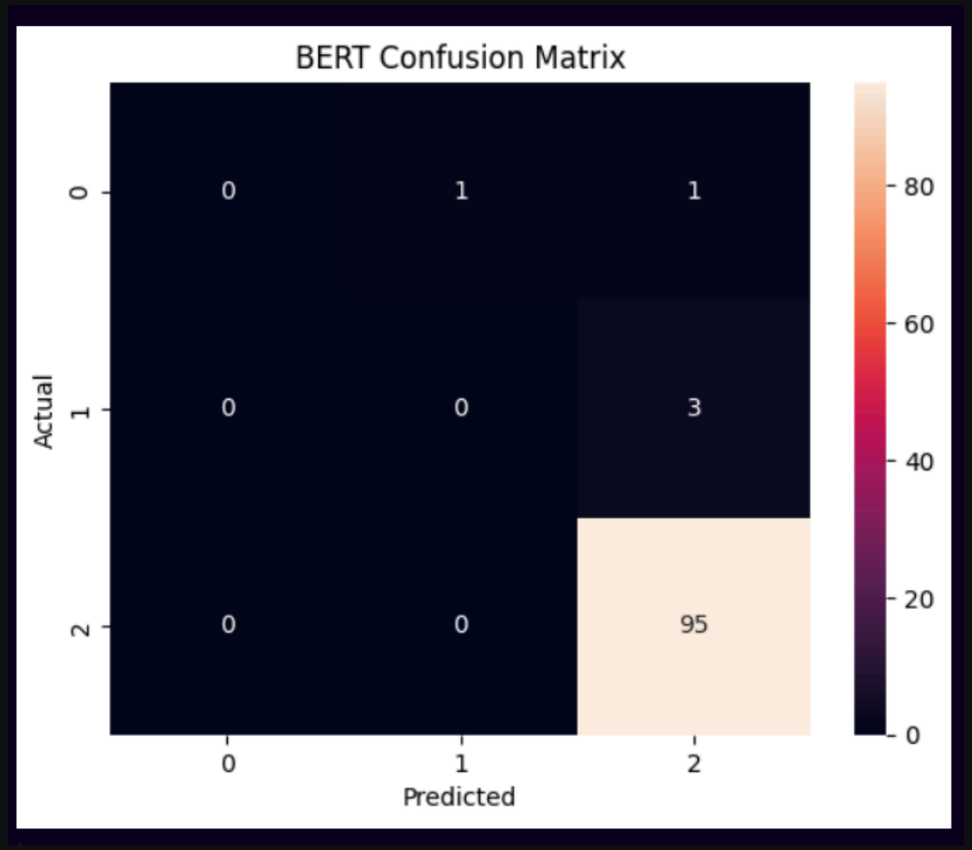
```
==================== TF-IDF + Logistic Regression ====================
Accuracy : 0.9617
Precision: 0.9248 (weighted)
Recall   : 0.9617 (weighted)
F1 Score : 0.9429 (weighted)

Classification Report:
              precision    recall  f1-score   support

           1       0.00      0.00      0.00         7
           2       0.00      0.00      0.00        16
           3       0.96      1.00      0.98       577

    accuracy                           0.96       600
   macro avg       0.32      0.33      0.33       600
weighted avg       0.92      0.96      0.94       600

Confusion Matrix:
[[  0   0   7]
 [  0   0  16]
 [  0   0 577]]

==================== BERT + Logistic Regression ====================
Accuracy : 0.9500
...
Confusion Matrix:
[[ 0  1  1]
 [ 0  0  3]
 [ 0  0 95]]
```

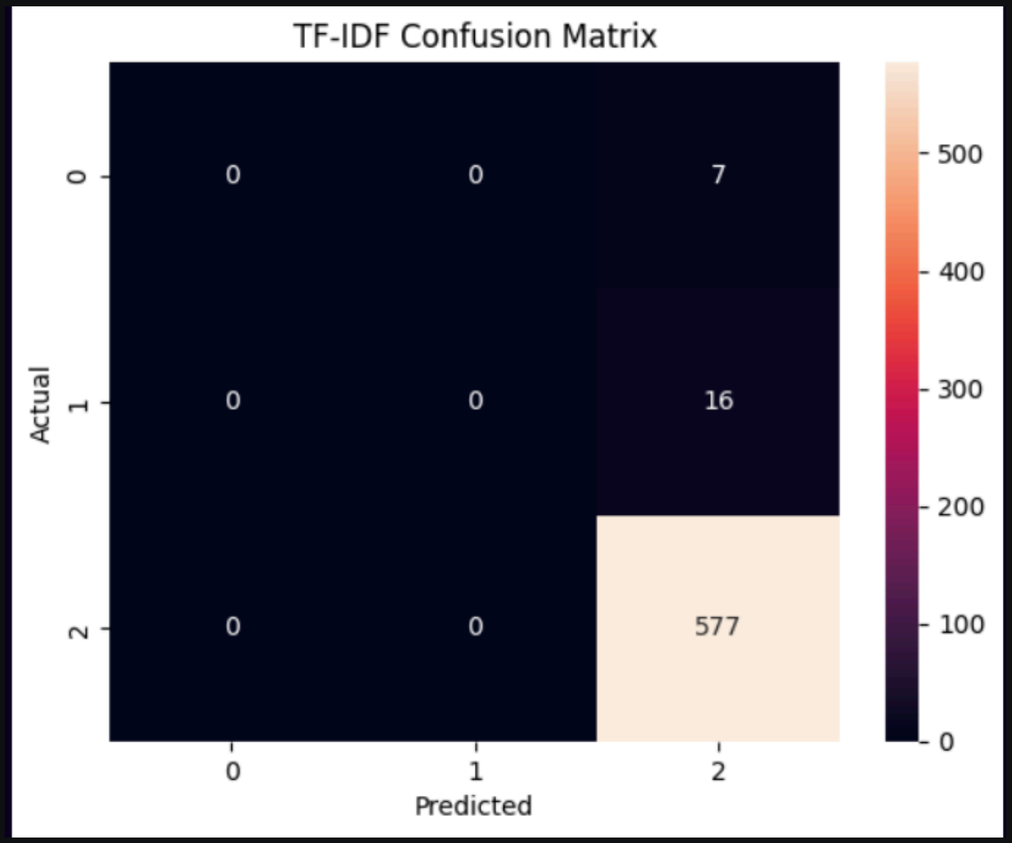
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Model Comparison



Confusion Matrix





Thank You